Linked Lists and Efficiency Solutions

Computer Science Mentors 61A

March 31, 2026 – April 3, 2026

1 Linked Lists

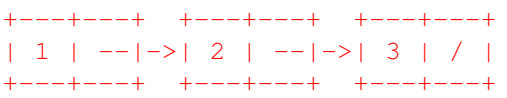
For each of the following problems, assume linked lists are defined as follows:

```
class Link:
    empty = ()
    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest is not Link.empty:
            rest_repr = ', ' + repr(self.rest)
        else:
            rest_repr = ''
        return 'Link(' + repr(self.first) + rest_repr + ')'

    def __str__(self):
        string = '<'
        while self.rest is not Link.empty:
            string += str(self.first) + ' '
            self = self.rest
        return string + str(self.first) + '>'
```

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

1. What will Python output? Draw box-and-pointer diagrams along the way.

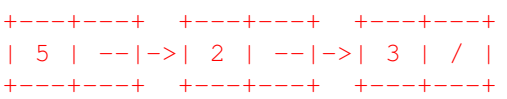
```
>>> a = Link(1, Link(2, Link(3)))
```



```
>>> a.first
```

```
1
```

```
>>> a.first = 5
```



```
>>> a.first
```

```
5
```

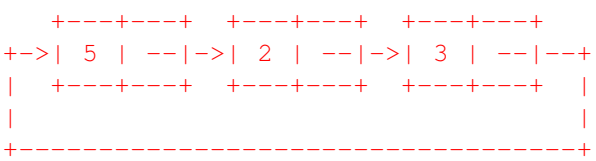
```
>>> a.rest.first
```

```
2
```

```
>>> a.rest.rest.rest.rest.first
```

```
Error: tuple object has no attribute rest (Link.empty has no rest)
```

```
>>> a.rest.rest.rest = a
```



```
>>> a.rest.rest.rest.rest.first
```

```
2
```

2. Write a function `skip`, which takes in a `Link` and skips every other element in the linked list.

   (a) First, implement `skip` non-mutatively. That is, return a new linked list with every other element skipped, and do not modify the original linked list.

```
def skip(lst):
    """
    >>> a = Link(1, Link(2, Link(3, Link(4))))
    >>> a
    Link(1, Link(2, Link(3, Link(4))))
    >>> b = skip(a)
    >>> b
    Link(1, Link(3))
    >>> a
    Link(1, Link(2, Link(3, Link(4)))) # Unchanged
    """
    if _____________________________________________:

        _____________________________________________

    elif _____________________________________________:

        _____________________________________________

    _____________________________________________


    if lst is Link.empty:
        return Link.empty
    elif lst.rest is Link.empty:
        return Link(lst.first)
    return Link(lst.first, skip(lst.rest.rest))
```

Base cases:

- When the linked list is empty, we want to return a new Link.empty.
- If there is only one element in the linked list (aka the next element is empty), we want to return a new linked list with that single element.

Recursive case:
All other longer linked lists can be reduced down to either a single element or empty linked list depending on whether it has odd or even length. Therefore, we want to keep the first element, and recurse on the element after the next (skipping the immediate next element with `lst.rest.rest`). To build a new linked list, we can add new links to the end of the linked list by calling skip recursively inside the `rest` argument of the `Link` constructor.

(b) Now, implement `skip` mutatively. That is, mutate the original list so that every other element is skipped. Do not call the `Link` constructor, and do not return anything.

```
def skip(lst):
    """
    >>> a = Link(1, Link(2, Link(3, Link(4))))
    >>> skip(a)
    >>> a
    Link(1, Link(3))
    """
```

```
def skip(lst): # Recursively
    if lst is Link.empty or lst.rest is Link.empty:
        return
    lst.rest = lst.rest.rest
    skip(lst.rest)

def skip(lst): # Iteratively
    while lst is not Link.empty and lst.rest is not Link.empty:
        lst.rest = lst.rest.rest
        lst = lst.rest
```

Because this problem is mutative, we should never be creating a new list - we should never have `Link(x)`, or the creation of a new Link instance, anywhere in our code! Instead, we'll be reassigning `lst.rest`.

In order to skip a node, we can assign `lst.rest = lst.rest.rest`. If we have lst assigned to a link list that looks like the following:

```
1 -> 2 -> 3 -> 4 -> 5
```

Setting `lst.rest = lst.rest.rest` will take the arrow that points form 1 to 2 and change it to point from 1 to 3. We can see this by evaluating `lst.rest.rest`. `lst.rest` is the arrow that comes from 1, and `lst.rest.rest` is the link with 3.

Once we've created the following list:

```
1 -> 3 -> 4 -> 5
```

we just need to call skip on the rest of the list. If we call skip on the list that starts at 3, we'll skip over the link with 4 and set the pointer from 3 to point to the link with 5. This is the behavior that we want! Therefore, our recursive call is `skip(lst.rest)`, since `lst.rest` is now the link that contains 3.

3. Fill in the implementation of `shuffle`.

```
def shuffle(lnk):
    """Swaps each pair of items in a linked list destructively and returns
       the modified linked list.

    >>> shuffle(Link(1, Link(2, Link(3, Link(4)))))
    Link(2, Link(1, Link(4, Link(3))))
    >>> shuffle(Link('s', Link('c', Link(1, Link(6, Link('a'))))))
    Link('c', Link('s', Link(6, Link(1, Link('a')))))
    """


    if lnk is Link.empty or lnk.rest is Link.empty:
        return lnk
    front = lnk.rest
    lnk.rest = shuffle(front.rest)
    front.rest = lnk
    return front
```

4. Write a recursive function `insert_all` that takes as input two linked lists, `s` and `x`, and an index `index`. `insert_all` should return a new linked list with the contents of `x` inserted at index `index` of `s`.

```
def insert_all(s, x, index):
    """
    >>> insert = Link(3, Link(4))
    >>> original = Link(1, Link(2, Link(5)))
    >>> insert_all(original, insert, 2)
    Link(1, Link(2, Link(3, Link(4, Link(5)))))
    >>> start = Link(1)
    >>> insert_all(original, start, 0)
    Link(1, Link(1, Link(2, Link(5))))
    """
    if ____________________ and ____________________:

        ____________________________________________

    if ____________________ and ____________________:

        ____________________________________________

    ________________________________________________


def insert_all(s, x, index):
    """
    >>> insert = Link(3, Link(4))
    >>> original = Link(1, Link(2, Link(5)))
    >>> insert_all(original, insert, 2)
    Link(1, Link(2, Link(3, Link(4, Link(5)))))
    >>> start = Link(1)
    >>> insert_all(original, start, 0)
```

```
Link(1, Link(1, Link(2, Link(5))))
>>> insert_all(original, insert, 3)
Link(1, Link(2, Link(5, Link(3, Link(4)))))
"""
if s is Link.empty and x is Link.empty:
    return Link.empty
if x is not Link.empty and index == 0:
    return Link(x.first, insert_all(s, x.rest, 0))
return Link(s.first, insert_all(s.rest, x, index - 1))
```

All of our return statements should return a new linked list.

Our base case should be the simplest possible version of the problem: when both `x` and `s` are empty, clearly the result is just the empty list.

We can now think of ways to break down this problem even further. Note that when the index to be inserted at is `0`, the problem is relatively easy: we just have to put all of the elements of `x` followed by all the elements of `s`. So the first element of the new list should `x.first`, and the rest of the new list should be `x.rest` concatenated with `s`, or `insert_all(s, x.rest, 0)`. Since we are using `x.first` and `x.rest`, we must check that `x` is nonempty to ensure that we do not error.

Finally, when the index to be inserted at is nonzero, we know that we're going to have some elements of `s`, then the elements of `x`, and then the rest of the elements from `s`. So the first element of the new list should be `s.first`. Then we can get the rest of the new list by inserting the contents of `x` at index `index - 1` of `s.rest`, reducing the index by 1 to account for the fact that we have removed the first element of `s`.

There's one issue we glossed over here: what if `x` is empty but `s` is not? Then we want to return the contents of `s`. But because the problem requires that we return a new linked list, we must recursively reconstruct `s` instead of simply returning it. You could add another base case to handle this, but as it turns out the second recursive case will handle this just fine since `Link(s.first, insert_all(s.rest, x, index - 1))` is just equivalent to `Link(s.first, s.rest)` when `x` is empty. Since the `x is not Link.empty` condition for the first recursive case will direct all situations where `x` is empty but `s` is not to the second recursive case, it turns out that we do not need to add anything else to this solution.

Convincing yourself that this problem works requires that you eventually reach a base case. Note that in either recursive call, we either reduce `s` or `x` by one element. So the base case will always eventually be reached, and the solution is valid.

2 Efficiency and Orders of Growth

An order of growth characterizes the runtime **efficiency** of a program as its input becomes extremely large. Since we care about rate of growth, we ignore constant coefficients and exclusively consider the fastest growing term. For example, on very large inputs, $2n^2+3n-20$ behaves the same as n^2. Common runtimes, in increasing order of time, are: constant, logarithmic, linear, quadratic, and exponential.

Examples:

Constant time means that no matter the size of the input, the runtime of your program is consistent. In the function `f` below, no matter what you pass in for `n`, the runtime is the same.

```
def f(n):
    return 1 + 2
```

A common example of a linear OOG involves a single for/while loop. In the example below, as `n` gets larger, the amount of time to run the function grows proportionally.

```
def f(n):
    while n > 0:
        print(n)
        n -= 1
```

We can modify this while loop to get an example of logarithmic OOG. Suppose that, instead of subtracting 1 each time, we halve the size of `n`. For `n = 1000`, the program would take 10 iterations to terminate (since $2^{10} = 1024$). The runtime is proportional to $\log(n)$.

```
def f(n):
    while n > 0:
        print(n)
        n //= 2
```

An example of a quadratic runtime involves nested for loops. For every one of the `n` iterations of the outer loop, there is `n` work done in the inner loop. This means that the runtime is proportional to n^2.

```
def f(n):
    for i in range(n):
        for j in range(n):
            print(i*j)
```

1. Give a tight asymptotic runtime bound for the following functions in $\Theta(\cdot)$ notation, or "Infinite" if the program does not terminate.

   (a)
   ```
   def one(n):
       while n > 0:
           n = n // 2
   ```

   $\Theta(\log n)$

   (b)
   ```
   def two(n):
       for i in range(n):
           for j in range(i):
               print(str(i), str(j))
   ```

   $\Theta(n^2)$

   (c)
   ```
   def three(n):
       i = 1
       while i <= n:
           for j in range(i):
               print(j)
           i *= 2
   ```

   $\Theta(n)$

2. Say we want to repeatedly insert some numbers to the end of a linked list.

   ```
   def append_many(s, items):
       for item in items:
           append(s, item)
   ```

   (a) Assuming `s` is initially length 1. How long will it take to complete the first insertion? The second? The nth?

   Notice that the list gets longer with each insertion, so each operation will make it harder to do the next. Therefore, the first insertion will take about 1 unit of time. The second will take about two units of time. The nth insertion will take n units of time.

   (b) Give the total runtime in $\Theta(\cdot)$ notation if `s` is initially empty and `items` contains n items.

   The total runtime will be the sum of all the inserts: $1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2} \in \Theta(n^2)$

3. Consider the following linked list function.

   ```
   def prepend(s, item):
       return link(item, s)
   ```

   (a) What does this function do?

   It takes in an existing `lst` and returns a new list with `item` at the front.

   (b) Assume `s` is initially length n, how long does it take to `prepend` once? `prepend` twice? `prepend` n times?

All inserts will take constant time. No matter how long the list is, it doesn't take any longer to add to the front. One insert will take one unit of time, and two will take roughly twice that. Therefore, the amount of time to do n inserts will be in $\Theta(n)$.